Tetris 2.0: Architecture

This document describes the big picture: what the system is made of, how the parts talk to each other, what happens at run time, where data lives, and how it is deployed. For the inside of each part (classes, algorithms, data formats), see `SOFTWARE_DESIGN.md`.

1. System Context

Tetris 2.0 is a desktop game. It runs on one player's computer and, only if the player turns it on, talks to a small score server.

```
flowchart LR
    Player([Player]) -- keyboard and mouse --> Game
    Game -- window and sound --> Player
    Game <--> Disk[(Local JSON files)]
    Game -. optional, HTTP + API key .-> Server[Score server]
    Server <--> ServerDisk[(cloud_scores.json)]
```

Actor or system	Role
Player	Plays, signs in and out, optionally runs the score server
Game	The desktop program (`tetris_ver2.py`)
Local files	Accounts, scores, and settings next to the game

2. Architectural Style

- **Single-process desktop application** with a classic **game loop** (read input, update, draw, repeat at 60 frames per second).
- **Layered inside the process:** rules at the center, with presentation, storage, and network layers around them.
- **Client and server** only for the optional score sync. The server is a separate small program.
- **Offline first:** every outside dependency (audio, files, network) has a safe fallback, so the game always starts and always plays.

3. Components

```
flowchart TB
    subgraph Game process: tetris_ver2.py
        Main[main and screen flow<br/>run_login, play]
        Login[LoginScreen<br/>accounts]
        Loop[Play loop _play<br/>input, timing]
        Rules[Game rules<br/>Game, Board, Piece, SevenBag]
        Draw[Presentation<br/>draw_* functions, themes]
        Audio[Audio<br/>SoundBank, Music]
        Store[Storage helpers<br/>users, scores, settings]
    end
    Sync[cloud_sync.py<br/>CloudSync client]
    Server[cloud_server.py<br/>Handler, ScoreStore]

    Main --> Login
    Main --> Loop
    Loop --> Rules
    Loop --> Draw
    Loop --> Audio
    Loop --> Store
    Login --> Store
    Draw -. reads state .-> Rules
    Store --> Sync
    Sync -. HTTP .-> Server
```

4. Layers

Layer	Contains	May use
Presentation and input	Draw functions, login layout, button hit testing, the play loop's event handling	Domain, Services
Application flow	`main`, `run_login`, `play`, save and sync decisions	Presentation, Domain, Services
Domain (rules)	`Piece`, `Board`, `SevenBag`, `Game`, scoring and kick tables	Only itself
Services	`SoundBank`, `Music`, storage helpers, `CloudSync`	Libraries, files, network

Rules never reach up. The only link from rules to the outside is the `game.events` list, which the loop reads each frame and passes to the audio service.

5. Runtime View

5.1 Start-up and screen flow

```
sequenceDiagram
    participant M as main()
    participant L as run_login
    participant P as play
    M->>M: pygame init, window, fonts, SoundBank
    loop until quit
        M->>M: apply light theme
        M->>L: show Sign In / Sign Up
        L-->>M: username, or None (Quit)
        alt username
            M->>P: play(username)
            P-->>M: "logout" or "quit"
        end
    end
    M->>M: pygame.quit()
```

5.2 One frame of play

6. Data Architecture

Data	Location	Format	Owner
Accounts	`users.json`	JSON object keyed by lowercase username: name, salt, hash, iterations	Accounts code
Player scores	`scores/<user>.json`	JSON list of up to 5 integers	Scores code
Player settings	`settings/<user>.json`	JSON object: `theme`, `muted`	Settings code
Server scores	`cloud_scores.json`	JSON object keyed by lowercase username: list of integers	`ScoreStore`

7. Cloud Sync Architecture

7.1 Overview

```
sequenceDiagram
    participant G as Game (_play)
    participant C as CloudSync client
    participant S as Score server
    participant D as cloud_scores.json
    G->>G: save score locally (top 5)
    G->>C: sync_scores(user, local list)
    C->>S: POST /sync/user  {"scores": [...]}  + X-API-Key
    S->>S: check key, path, body size, values
    S->>D: merge with stored list, keep top 5, save
    S-->>C: {"scores": merged}
    C-->>G: merged list
    G->>G: write merged list to local file, show HIGH SCORES (CLOUD)
```

When does it run: at game over, on Sign Out, and on Quit from the pause menu or after game over (not on Restart).

8. Concurrency Model

Part	Model
Game	Single thread. One loop does input, rules, audio calls, and drawing
Audio	pygame's mixer plays sounds in its own audio thread; the game only starts, pauses, and stops channels
Cloud sync	Blocking call on the game thread, up to 1.5 seconds, and only at save points
Server	One thread per request, with a lock on the data file

9. Deployment View

```
flowchart LR
    subgraph Player's computer
        A[tetris_ver2.py<br/>cloud_sync.py] --> B[(users.json<br/>scores/<br/>settings/)]
    end
    subgraph Any reachable computer, optional
        C[cloud_server.py] --> D[(cloud_scores.json)]
    end
    A -. TETRIS_CLOUD_URL + KEY .-> C
```

Item	Details
Runtime	Python 3 and pygame on the player's computer
Game files	`tetris_ver2.py` and `cloud_sync.py` in the same folder
Server files	`cloud_server.py` and `cloud_sync.py` in the same folder
Server dependencies	Python standard library only

10. Technology Choices

Choice	Reason
Python 3	Readable, fast to build and test, runs everywhere
pygame	Simple window, input, drawing, and sound for a 2D game
JSON files	Human readable, no database to install
Standard library HTTP server and client	No extra packages for the optional sync
PBKDF2-HMAC-SHA256	Standard password hashing available in Python without extra packages
Generated audio	No audio files to package; works the same everywhere

11. Quality Attributes

Attribute	How the architecture supports it
Testability	Rules are pure logic; the server runs on a free port in tests; dummy video and audio drivers allow headless runs
Reliability	Validated loads, atomic server saves, and fail-soft audio and network
Security	Hashed passwords, validated names and inputs, constant-time key check, size limits, local-only server by default, network off by default
Performance	Light per-frame work; music loops are built once per tempo and cached; clamped gravity
Portability	Pure Python plus pygame; no platform-specific code
Modifiability	Rule numbers are constants; new sounds, themes, or kick tables are data entries; the server and client are separate files

12. Extension Points

To add...	Change
A new theme	Add an entry to `THEMES` and `THEME_ORDER` (each theme must color all 7 pieces; a test checks it)
A new sound	Add a name to `SOUND_NOTES`, then append that name to `game.events` where it happens
A new scoring rule	Add a constant near the scoring tables and use it in `_lock_active_piece`
A new power-up	Add state to `Game`, an action method like `use_bomb`, a key in `_play`, a HUD line in `draw_hud`
A new API route	Add a branch in `Handler` and a method in `ScoreStore` and `CloudSync`

13. Risks and Constraints

Risk or constraint	Mitigation or note
Large single game file	Sections are clearly labeled; splitting into modules is the next refactor
Global theme variables	Tests reset the theme after use; a Theme object would be cleaner
Blocking sync	Short timeout; background thread is a possible improvement
JSON files and many users	Fine for a few local players; a database would be needed for more
Shared API key	Use `https` outside a trusted network
Unfinished run lost when the	Use Pause > Quit or Sign Out; saving on the quit